Modèles Algorithmiques Fondamentaux : Structures de Données et Stratégies de Parcours

Thème principal : Initiation au développement (C++) - Algorithmique de base

Synthèse exécutive : Ce cours pose les fondements méthodologiques indispensables de l'algorithmique. Avant d'écrire la moindre ligne de code, l'informaticien doit analyser le contrat d'interface et le mécanisme d'accès de sa structure de données (Direct, Séquentiel ou Dynamique). Le cours formalise trois grands modèles algorithmiques canoniques applicables aux structures à accès direct (tableaux, chaînes de caractères) : (1) le parcours complet avec traitement systématique, (2) le parcours complet avec traitement conditionné, et (3) la recherche de première occurrence. Sont également détaillés : la gestion rigoureuse des bornes d'indices (de 0 à N-1 pour éviter les erreurs de dépassement 'out of bounds'), le principe du pessimisme algorithmique, l'obligation incontournable d'une double condition d'arrêt pour toute recherche, ainsi que le comparatif précis entre les deux variantes d'implémentation d'une recherche selon que l'on vise la simplicité syntaxique ou la conservation de la position indicée exacte.

Les Mécanismes d'Accès aux Structures de Données

L'étape zéro de toute conception algorithmique consiste à analyser le contrat d'interface offert par la structure de données sous-jacente. Si la structure propose un accès direct (comme un tableau statique ou une chaîne de caractères), chaque case mémoire est immédiatement adressable par son indice. En revanche, un flux séquentiel (tel qu'un fichier texte brut ou une liste chaînée simple) n'expose que trois opérations primitives : se positionner au début, avancer d'un pas vers le suivant, et tester si la fin est atteinte. Tenter d'utiliser une boucle itérative indicée de type 'T[i]' sur un flux séquentiel constitue une erreur conceptuelle rédhibitoire qui invalide totalement l'algorithme.

- Structure de données : Collection regroupant un ensemble d'éléments de même type.
- Accès direct : Atteindre instantanément n'importe quel élément via son indice ou sa clé (Complexité O(1)).
- Accès séquentiel : Contrainte d'avancer pas à pas du début vers le suivant (début, suivant, fin).
- Incompatibilité structurelle : Interdiction absolue d'appliquer un modèle d'accès direct (ex: T[i]) sur un flux purement séquentiel.

Cas concret enseignant : *Le professeur met en garde contre l'erreur classique observée en contrôle : tenter d'écrire 'T[i]' sur un fichier CSV ou un fichier texte ouvert. Comme le système d'exploitation ne fournit qu'un curseur séquentiel ligne à ligne, un tel code est sémantiquement faux et conduit directement à un zéro sur la question.*

Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique dès qu'aucune exclusion n'est permise entre deux bornes définies. Il répond à des problématiques telles que : afficher toutes les valeurs d'un tableau, initialiser chaque case à zéro, ou convertir chaque caractère d'une chaîne en majuscule. En pseudo-code normalisé, pour un tableau de taille N, l'indice i démarre à 0 et termine impérativement à N - 1. Le traitement est appliqué aveuglément à chaque tour de boucle sans test préalable.

- Objectif : Appliquer un traitement uniforme à 100% des éléments situés entre la borne de début et la borne de fin.
- Structure itérative : Utilisation obligatoire d'une boucle bornée 'Pour' (nombre d'itérations calculable à l'avance).
- Gestion des bornes : Pour N éléments indicés dès 0, la boucle varie strictement de 0 à N - 1.
- Risque de débordement : Boucler jusqu'à N engendre une exception critique de dépassement d'indice (out of bounds).

Cas concret enseignant : *Exemple concret donné en amphi : 'Afficher un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente des débutants est d'écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat à l'exécution lors de la tentative de lecture de la case mémoire T[N] qui n'appartient pas au tableau.*

Second Modèle : Le Parcours Complet avec Traitement Conditionnel

Le modèle du parcours complet conditionné conserve l'inspection systématique de l'ensemble des éléments, mais protège l'exécution de l'action de traitement par une condition logique. Il est utilisé pour compter des occurrences particulières (ex: compter les voyelles dans un texte), filtrer des données (ex: afficher uniquement les entiers pairs) ou accumuler des valeurs spécifiques. Si la condition implique des éléments adjacents (ex: T[i] > T[i-1]), le concepteur doit impérativement réajuster les bornes de la boucle.

- Parcours exhaustif : Tous les éléments de la zone sont inspectés du début à la fin.
- Traitement sélectif : L'action n'est déclenchée que si l'élément inspecté valide un prédicat logique.
- Architecture : Imbrication d'une structure alternative 'Si ... Alors' au sein d'une boucle bornée 'Pour'.
- Conditions complexes : Comparaison possible avec des éléments adjacents (i-1 ou i+1) en ajustant les bornes.

Cas concret enseignant : *L'analogie de la chaîne de montage en usine : 'Chaque pièce défile obligatoirement devant le contrôleur qualité (parcours complet systématique d'inspection), mais le contrôleur n'appose le tampon de rebut (action de traitement) que si et seulement si la pièce présente une fissure (traitement conditionné).'*

Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

Contrairement aux deux premiers modèles, la recherche de première occurrence stoppe l'itération dès que la cible est identifiée. L'informaticien doit adopter une démarche pessimiste en postulant que l'élément n'existe pas. Un algorithme de recherche valide DOIT posséder deux conditions d'arrêt : (1) le succès de la recherche, (2) l'épuisement complet de la collection.

- Pessimisme algorithmique : Initialisation systématique d'un indicateur booléen 'trouvé' à Faux.
- Itération non bornée : Utilisation obligatoire d'une boucle 'Tant que' (nombre de répétitions inconnu a priori).
- Double condition d'arrêt : Arrêt dès que l'élément est trouvé OU dès que la structure a été entièrement explorée.
- Variante 1 (Rupture immédiate) : Interruption nette, conserve l'indice i exact.
- Variante 2 (Tant que global) : Test unique en tête ('Tant que i < N et NON trouvé'), plus simple syntaxiquement.

Cas concret enseignant : *Devise du prof : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans votre collection, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle que tout algorithme de recherche présenté en examen avec une seule condition d'arrêt est sanctionné par une note éliminatoire.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, du moment que la structure offre des primitives permettant l'accès indicé ou un parcours direct de type tableau, mais il faut toujours s'assurer que les bornes dynamiques sont bien connues et mises à jour.

Question étudiant : Pourquoi s'embêter à présenter deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : La variante 1 est indispensable si vous avez besoin de récupérer l'indice exact de l'élément trouvé pour la suite du programme. La variante 2 est plus facile à écrire et évite les erreurs de gestion de curseur si vous voulez juste savoir si l'élément est présent ou absent.

Question étudiant : Comment fait-on si la condition de recherche dans un parcours conditionnel dépend de l'élément précédent ou suivant ?
Réponse du professeur : Il faut impérativement décaler les bornes de la boucle 'Pour'. Par exemple, si vous comparez T[i] à T[i-1], la boucle doit démarrer à i = 1 et non à i = 0 pour éviter un accès mémoire invalide à l'indice -1.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut pas utiliser la recherche de première occurrence (qui s'arrête au premier trouvé), mais bien le **parcours complet avec traitement conditionné**, où l'action consiste à enregistrer ou afficher l'indice i chaque fois que T[i] == 5.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments, en explicitant le modèle retenu.

Résolution pas à pas :
Étape 1 : Analyse du problème et sélection du modèle. Le problème exige d'inspecter l'ensemble des éléments d'un tableau (structure à accès direct) de la première à la dernière case. L'action d'affichage ne doit être exécutée que si l'élément respecte une propriété arithmétique (valeur > 0). Le modèle canonique est donc le 'Parcours complet avec traitement conditionné'. Étape 2 : Définition des bornes d'indices. Le tableau comporte N cases. En convention informatique standard à indexation zéro, la première case est à l'indice 0 et la dernière case est à l'indice N - 1. Étape 3 : Choix de la structure de contrôle. Nombre d'itérations connu à l'avance -> boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Définition du prédicat logique et du traitement. À chaque itération, on évalue la condition 'T[i] > 0'. Si cette assertion est vraie, on déclenche l'instruction 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire didactique : Cet algorithme assure l'exhaustivité du traitement sans risque de débordement mémoire (l'indice i s'arrête strictement à N-1) et filtre sélectivement les valeurs requises.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on employer pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : Le troisième modèle : la recherche de première occurrence (avec une boucle 'Tant que' et une double condition d'arrêt).

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il obligatoirement N - 1 et non N ?

R : Parce que les tableaux en informatique commencent à l'indice 0. Un tableau de N éléments possède donc des indices allant de 0 à N - 1. Tenter d'accéder à l'indice N provoque une erreur de dépassement de mémoire (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure de contrôle d'une recherche de première occurrence ?

R : Exactement deux conditions d'arrêt minimales : (1) l'élément a été trouvé, ou (2) la collection a été entièrement explorée sans succès.